Syrudas AI: A Local-First AI Workspace with Pluggable Model Providers

Version 0.7.3 · July 2026 · Len · MIT License

Abstract

Syrudas AI is a self-hosted AI workspace for Windows that runs entirely on the user's own machine. It offers streaming chat, an autonomous agent mode with tool use and Model Context Protocol (MCP) support, durable cross-conversation memory, local retrieval over the user's own files, a deep-research pipeline, a blind model-comparison arena, an AI-assisted writing editor, a hardware-aware model cookbook, and editor integration. What makes this breadth tractable for a single maintainer is a deliberate architectural bet: **the model is a plugin.** Every backend — local weights or a hosted frontier API — reaches the application through one small contract, and nothing above that contract knows which model is answering. This paper is a from-first-principles account of the system: the thesis, the abstraction that carries it, the normalized data model that makes the abstraction cheap, how each feature falls out of that model as a consequence rather than a bolt-on, and the security posture that a local, single-user tool can honestly promise.

1. Motivation

Using a large language model increasingly means renting a seat in a vertically integrated stack: one vendor's model, behind one vendor's application, with the user's conversations, keys, and files living on one vendor's servers. That arrangement is convenient and, for many, sufficient. It is also fragile in a particular way: it couples the *interface* you rely on to the *model* you happen to prefer this quarter, and it couples both to a third party's continued goodwill about your data. Projects such as PewDiePie's **Odysseus** (2026) showed that a meaningful audience wants the opposite: a workspace you run yourself, where the conversation history, the API keys, and the documents never leave the machine.

Syrudas AI begins from that local-first premise but pushes on one idea harder than most such projects do. Model backends are the fastest-moving, least-durable part of this ecosystem — new weights and new APIs arrive monthly, and today's best choice is next quarter's fallback. A workspace built around any specific model, or any specific provider's SDK, inherits that churn. So the organizing principle here is that **the model is the most replaceable component, and the architecture should treat it that way.** Everything the user actually values — their history, their tools, their documents, the agent runtime, the UI — sits above a thin seam, and the model plugs into that seam.

Five concrete goals follow from this stance:

1. **Any model, one workspace.** Local weights via Ollama or LM Studio, and hosted models via OpenRouter or first-party APIs, share one UI, one conversation store, and one agent runtime.
2. **Local-first and private by construction.** The server binds to `127.0.0.1` and rejects non-loopback requests. There is no account, no telemetry, and no cloud component. Keys are stored locally and

masked in every response.

3. **Agentic, but consent-gated.** The model can plan and act — run commands, read and write files, search and read the web, remember facts, search indexed documents — with an explicit, per-action human gate on the dangerous parts.
4. **A hub, not a silo.** The workspace speaks the OpenAI wire dialect in both directions, so external tools can use its configured models as a service.
5. **Shippable to a non-developer.** A single portable executable that unzips and double-clicks, with zero-configuration onboarding when a local model server is already present.

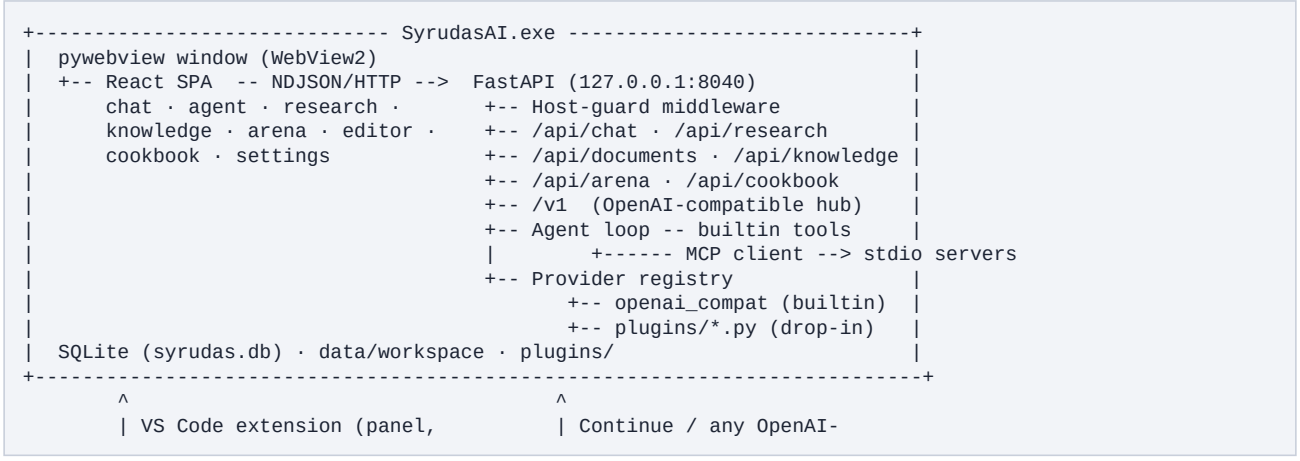
The rest of this paper traces how a single abstraction, enforced everywhere, lets those goals coexist inside a codebase one person can hold in their head.

2. System overview

Syrudas is a two-tier application with a deliberately small surface between the tiers. The **backend** is Python 3.13 on FastAPI: it serves a REST and streaming API, owns a single SQLite database, runs the agent and research loops, and hosts the provider, MCP, and retrieval subsystems. The **frontend** is a React 19 / TypeScript single-page app built with Vite — chat, editor, arena, cookbook, and settings — compiled to static assets that the backend serves. The same compiled bundle runs in three places without modification: an ordinary browser tab, the packaged desktop window, and a VS Code panel.

The packaged desktop application wraps both tiers in a native window (pywebview over the Windows WebView2 runtime), with the FastAPI server running on a background thread of the same process. There is no second process to supervise and no port coordination beyond a single fixed local port; closing the window stops the server, and launching the executable again while it is running simply opens a new window onto the existing instance.

State is intentionally boring, which is a feature rather than an apology. One SQLite file holds conversations, messages, provider instances, MCP registrations, memories, indexed-document chunks and their embeddings, editor documents, arena results, and settings. One workspace folder holds files the agent writes. In the portable build both live directly beside the executable, so copying the folder copies the entire installation and deleting it removes every trace — no registry keys, no scattered application-data directories. Lightweight session state — the open view and conversation, plus the model and theme choices — is kept in the frontend's local storage (persisted across desktop restarts by the WebView2 storage path), so the app reopens exactly where the user left off.



```
        | ask-about-selection)              | compatible client via /v1
```

3. The provider contract

Everything in the previous section rests on a single class. A provider *type* is a Python class; a provider *instance* is that class configured with user data — a base URL, an API key — in the settings UI and persisted in SQLite. The entire contract is five methods, two of them optional:

```
class ModelProvider(ABC):
    type_id: str                        # "openai_compat", "anthropic", ...
    display_name: str
    config_fields: list[ConfigField]    # declares the settings form to render

    async def list_models(self) -> list[ModelInfo]: ...
    async def chat(self, model, messages, tools=None, params=None)
        -> AsyncIterator[StreamEvent]: ...
    async def embed(self, model, texts) -> list[list[float]]: ...   # optional
    async def check(self) -> str: ...                               # optional
```

Three decisions turn this small surface into the load-bearing wall of the whole system.

A normalized interior. Messages, tool specifications, tool calls, and stream events are defined exactly once, in an OpenAI-flavored shape (Section 4). A provider adapter's only job is to translate between that shape and its backend's wire format, at the edge. The chat pipeline, the agent loop, the persistence layer, and the UI are all written against the normalized types and never learn which backend produced them. Adding support for a new API is therefore a self-contained translation file, not a change that ripples through the application.

Streaming as the only mode. `chat()` returns an async iterator of stream events; there is no separate "get a completion" method. A non-streaming response is simply a stream that ends quickly. This collapses what is usually two code paths — with two sets of bugs — into one, and it means every consumer, from the chat view to the deep-research synthesizer, handles partial output the same way.

Discovery by drop-in. The registry imports its builtin adapters statically, because a PyInstaller onefile bundle is invisible to `pkgutil`'s directory scanning and dynamic-only imports would never be packaged at all. On top of that, it scans a user-writable `plugins/` folder next to the executable at startup. The consequence is unusual and, for a local tool, delightful: a user can teach the *packaged* application to speak to a brand-new backend by dropping a single Python file beside the exe and restarting — no rebuild, no toolchain.

The single builtin adapter, `openai_compat`, is enough for most of the ecosystem, because Ollama, LM Studio, the llama.cpp server, vLLM, OpenRouter, and OpenAI itself all speak the same dialect. Optional connectors for **Anthropic (Claude)** and **Google (Gemini)** ship as drop-in plugins that translate those non-OpenAI dialects. The `embed()` method is the one place the contract grew a new capability rather than a new backend: providers that implement it become eligible to power the retrieval subsystem (Section 8), while providers that do not simply raise `NotImplementedError` and are omitted from the embedding-model picker.

4. The normalized interior

The abstraction in Section 3 is only cheap because the types it exchanges are few and stable. They live in one module and are shared verbatim by every provider, route, and persistence call:

- `Message` — a role (`system` / `user` / `assistant` / `tool`), text

content, an optional list of tool calls (on assistant messages), and an optional tool-call id (on tool-result messages).

- `ToolSpec` — a tool offered to the model: name, description, and a

JSON-Schema parameter object.

- `ToolCall` — a model's request to invoke a tool: id, name, parsed

arguments.

- `StreamEvent` — one event in a provider's output stream. The vocabulary

is small and closed: `text_delta`, `tool_call`, `usage`, `error`, `done`.

The event vocabulary is where the "streaming as the only mode" decision pays off structurally. The chat route serializes these events to the client as newline-delimited JSON, but along the way it *interleaves* events the provider never produced — an `approval_required` when the agent needs consent, a `research_status` as the research pipeline works, a `tool_result` after a tool runs. Because the client already consumes one flat event stream, plain chat, agent tool activity, research progress, and consent prompts all arrive over the same channel and are rendered by the same reader. A new interactive behavior is usually a new event type, not a new transport.

Choosing an OpenAI-flavored shape for the interior was pragmatic rather than ideological: most backends already speak it, so most adapters are nearly identity functions, and the `/v1` hub (Section 12) can re-emit the interior almost directly. The cost is that genuinely non-OpenAI providers (Anthropic's content-block model, Gemini's `parts`) do real translation work — but that work is confined to their adapter, which is exactly where the contract says it belongs.

5. The chat pipeline

`POST /api/chat` accepts a message, persists it, and returns a streamed NDJSON body: one JSON event per line, consumed on the client by a `ReadableStream` reader. NDJSON-over-POST was chosen over two obvious alternatives for concrete reasons. WebSockets bring a connection lifecycle — reconnection, heartbeats, state — that a request/response interaction does not need. Server-Sent Events cannot carry a POST body, which a chat turn requires. A streamed POST response is the smallest thing that works, and it composes cleanly with the interleaved event model of Section 4.

Two robustness properties are worth naming because they recur throughout the system. First, **messages are persisted as they complete**, so a conversation survives an interrupted stream, an application restart, or a version upgrade; the partial assistant text produced before a mid-stream failure is still written. Second, each conversation carries a **generation counter**. When history is rewritten out of band — a rewind, an edit-and-resend, a delete — the counter is bumped, and any still-running stream checks it before persisting. This is the mechanism that stops a "zombie" stream (client gone, cancellation not yet observed) from orphaning tool messages or resurrecting rows in a conversation the user already deleted. The same guard protects the research pipeline, which also persists into a conversation.

Treating the model as a plugin has a consequence for conversation state that is easy to miss: a workspace whose model is interchangeable will accumulate threads answered by *different* models. A conversation

therefore records the provider and model it last used, and reopening one restores that pick. Without it, a global picker silently applies whatever model was last chosen anywhere to a thread every previous turn answered with another — a wrong-model reply that leaves no trace in the transcript. The same persistence instinct applies to the `usage` event: the token counts a provider reports at the end of a turn are stored on the assistant message they belong to, so per-reply accounting survives a reload instead of living only in the open tab. In agent mode, where one turn may make several provider calls, counts are recorded per step against the message that step produced.

Long conversations are trimmed to fit the model's context before each turn: the system prompt and the newest messages always survive, the oldest turns fall off first, and a tool result whose originating tool call was trimmed away is dropped rather than sent as a dangling reference that confuses backends.

6. Agent mode

Agent mode turns a conversation into a plan-act loop. The model receives the tool schemas alongside the conversation; any tool calls it returns are executed, their results appended as `tool` messages, and the loop repeats until the model stops calling tools or a step ceiling (fifteen) is reached. The step ceiling is a blunt instrument, but for a local tool it is the right kind of blunt: it guarantees termination without trying to be clever about what "stuck" means.

The builtin tools, and how each is gated, are the heart of the agent's safety posture:

Tool	Capability	Guard
`shell`	Run a PowerShell command	**Per-call human approval**
`file_read` / `file_list`	Read and list text files	Path sandbox
`file_write`	Write a text file	Free in workspace; **approval** outside it
`web_search`	DuckDuckGo search	—
`web_fetch`	Fetch a URL as readable text	**Per-call approval**; refuses private IPs
`memory_save` / `memory_delete` / `memory_search`	Durable facts	Ungated, fully user-visible
`knowledge_search`	Retrieve indexed passages	Ungated, read-only

The approval gate. The one-way NDJSON stream cannot pause for a dialog box, so consent arrives out of band. When the agent proposes a gated call, the loop emits an `approval_required` event carrying a fresh approval id, then parks on an `asyncio.Future` registered under that id. A separate endpoint, `POST /api/approvals/{id}`, resolves the future when the user clicks Approve or Deny in the UI; the loop wakes and either runs the tool or reports "the user denied this tool call" as an ordinary tool result — so a denial redirects the model rather than crashing the run. Approvals have a timeout so an abandoned run cannot park forever, and they are single-shot: an unknown or already-used id is rejected. There is deliberately no "always allow."

Per-call, argument-aware gating. Gating is not a static property of a tool but a decision made per invocation, through a `needs_approval(args)` hook. This is what lets `file_write` be frictionless inside the

agent's own workspace yet require approval the moment its resolved target lands in a user-granted external folder — the same tool, two risk profiles, distinguished by the actual arguments.

The file sandbox. Relative paths resolve inside a dedicated workspace folder. Absolute paths are honored only inside folders the user has explicitly granted under Settings; every other path is refused with an error string the model can read and adapt to. Grants are data, surfaced to the model in its system prompt so it knows where it may work, and revocable at any time. Crucially, paths are re-resolved to their real location before the check, so a symlink or junction cannot be used to step outside a granted root.

MCP. Users can register stdio MCP servers by command line. Their tools are namespaced by server (`filesystem_read_file`) and merged into the agent's toolset, indistinguishable to the model from builtins. One implementation subtlety earned its place in the code comments: each server connection is owned by a single dedicated asyncio task, because anyio cancel scopes must be entered and exited by the same task; other tasks only use the already-initialized session object.

7. Persistent memory

An agent that forgets everything between conversations is a weaker assistant than the model's weights allow. Memory addresses this with deliberate modesty. The `memory_save` tool records a short, distilled fact; the newest memories, within a character budget, are injected into the agent's system prompt at the start of each agent-mode conversation, so preferences, project context, and decisions carry forward. `memory_search` reaches the older ones the budget omits, and `memory_delete` removes what has gone stale.

Memory is intentionally ungated, and the reasoning is worth stating because it runs opposite to the caution elsewhere. A saved memory has no effect outside the application; every save appears in the transcript as a tool card; and Settings → Agent memory is an always-available surface to review, add, and delete entries by hand. The combination — no external effect, full visibility, a standing kill switch — makes a per-save approval prompt pure friction. Plain, non-agent chat never receives memories at all, so the feature is scoped to the mode that opts into agency. Memories ride only on the request-local system prompt; they are never baked into a conversation's stored prompt, so deleting a memory truly forgets it everywhere.

8. Knowledge: local retrieval

Retrieval lets the workspace answer from documents far larger than any context window, without those documents ever leaving the machine — the local-first promise applied to the user's own corpus. Files or folders (text, code, PDFs) are indexed under Settings → Knowledge. Each source is extracted to text, chunked with a paragraph-aware sliding window that overlaps adjacent chunks so a fact split across a boundary is still retrievable, embedded through a provider that implements `embed()`, and stored as normalized `float32` vectors directly in SQLite alongside the chunk text.

The search itself is deliberately unsophisticated: embed the query, then compute a cosine similarity against every stored chunk and return the top matches. At the few-thousand-chunk scale this application targets, a brute-force dot product in pure Python completes in milliseconds, which buys a real architectural simplification — no vector database to run or bundle, no numpy dependency, and no index to keep consistent. The retrieval quality ceiling is lower than a dedicated engine's, but for "chat with my folder of

notes" it is comfortably sufficient, and the cost is a few dozen lines.

Two properties keep it safe and honest. Indexing is sandboxed to the same allowed roots as the file tools, re-resolving each walked file so a junction or symlink cannot pull outside-the-sandbox content into the index. And changing the configured embedding model clears the index, because vectors produced by different models are not comparable and silently mixing them would return plausible-looking nonsense. Retrieval surfaces to the agent as the read-only `knowledge_search` tool and is reused, as the next section describes, by deep research.

9. Deep research

Deep Research answers a question by reading the web and the local index, then writing a cited report. The notable design choice is that it is a **deterministic pipeline, not an autonomous agent loop**: plan, then gather, then read, then synthesize, as fixed stages. The reason is reliability with the smaller local models this application often runs. Asking a 7-billion-parameter model to correctly drive a fifteen-step tool loop toward a good report is a gamble; asking it to perform four well-scoped single-shot tasks is not. One completion turns the question into a handful of search queries; each query runs through web search and the candidate URLs are deduplicated; the top sources are fetched to readable text (under the same private-address protection as the agent's `web_fetch`) and the local knowledge index is consulted; a final streamed completion writes a Markdown report that cites its numbered sources, followed by a Sources list.

Because the fetched pages are untrusted text fed into a prompt, the synthesizer is hardened against injection: each source body is wrapped in explicit fenced delimiters and the system prompt states that fenced content is data to be summarized, never instructions to obey; source titles are sanitized before they enter the report so a crafted result cannot smuggle a Markdown image beacon or link. The whole run persists as an ordinary conversation, which means history, export, and the sidebar work with no new machinery — a recurring dividend of routing new features through the existing normalized model.

10. The writing editor

The editor is a local document workspace with AI editing folded in. Documents autosave to SQLite. A user selects text and applies a preset action (Improve, Shorten, Expand, Fix grammar), continues from the cursor, or issues a custom instruction; the model's suggestion streams into a panel to accept or reject. The edit endpoint, `POST /api/documents/edit`, is stateless — it returns only the replacement text and never mutates a document server-side. The client owns the decision to splice an accepted suggestion into the document at the captured selection, which keeps the server a pure text transformer and the document the single source of truth in the browser.

Two implementation details reflect hard-won correctness. Autosave is a dirty-flag debounce backed by a stale-proof mirror of the loaded document, and it *flushes* rather than merely cancels when the user switches documents or leaves the editor — so a fast edit-then-switch cannot silently drop the last change. And the text area is locked while a suggestion is pending, because an accepted suggestion is spliced at offsets captured when the run started; allowing edits in between would let those offsets drift and land the replacement in the wrong place. Both are the kind of bug that only appears under real use, and both were closed before the feature shipped.

11. The blind arena

The arena exists to answer a question the multi-provider core makes cheap to ask: which model is actually better for *my* prompts? It runs one prompt against two chosen models with their identities hidden — a coin flip decides which model fills which on-screen column, so position never leaks identity. Both answers stream in parallel through two calls to `/api/complete`, a stateless single-shot completion endpoint that the arena shares with any future feature needing a one-off generation. The user votes — A, B, tie, or both bad — which reveals the names and records the outcome to a local per-model win/loss/tie leaderboard. Almost all of the feature is a view; the routing that makes it possible was already built for chat.

12. The model cookbook

The cookbook is the one subsystem that reaches slightly outside the "model is a plugin" frame, and it does so carefully. It detects local hardware — CPU, RAM, and GPU VRAM, the last via `nvidia-smi` with a WMI fallback — using only the standard library and degrading every probe to "unknown" rather than raising, so a machine it does not fully understand still gets a usable page. It then rates a curated catalog of models as *fits your GPU* / *tight* / *CPU* / *too big* for that hardware, using rough per-model footprint estimates presented honestly as estimates. Models can be downloaded straight into Ollama with streamed pull progress and removed again.

The design keeps the plugin philosophy intact by being strictly *additive*: the cookbook never becomes the way models are served or selected. It only helps get weights onto disk via Ollama's native API, after which those models appear through the ordinary provider and the normal model picker. Downloading is therefore Ollama-specific — other backends get recommendations but not one-click installs — which is an honest limitation rather than a hidden coupling. Hardware detection, which shells out to external processes, runs off the event loop so a cookbook page load can never freeze a concurrent chat or research stream.

13. Interoperability

Syrudas is both a *client* of the OpenAI dialect, through `openai_compat`, and a *server* of it. The `/v1` surface exposes every model of every configured provider under a namespaced id (`ollama-local/llama3.1:8b`), and `chat/completions` translates each request into the normalized interior and routes it to the right backend, with streaming, tool calls, and usage preserved. These calls are stateless and never touch conversation history.

This inverts the usual integration burden. Rather than Syrudas writing an adapter for every editor and tool that might want to use it, any OpenAI-compatible client adopts Syrudas as its backend by changing one base URL — and in doing so gains uniform access to every model the user has configured, local and hosted alike, behind a single endpoint. The Continue extension for VS Code is the reference consumer. A dedicated VS Code extension complements the hub: it embeds the full workspace UI in an editor panel and adds a right-click *Ask About Selection* command that opens a chat prefilled with the selected code via a `?prompt=` deep link.

14. File attachments

Attachments follow a stateless pipeline that mirrors the rest of the system's preference for keeping state in one place. The client uploads a file to `POST /api/attachments`; the server extracts text — UTF-8 for text-like formats, pypdf for PDFs, binaries rejected, with a character cap and explicit truncation flags — and returns it. The client then embeds that text into the outgoing message inside `<file name="...">`

delimiters.

Storing attachment content *inside the message* rather than in a side table of blobs buys three properties at once: replaying history to any provider needs no special casing, because the file is just message text; backup and export are simply the database file; and the UI reconstructs collapsible per-file chips from the message text alone. The trade-off is larger message rows, which is acceptable at the capped sizes. For documents too large for the model's context window, the knowledge subsystem (Section 8) is the better path, and the two features compose: attach for a one-off, index for a corpus.

15. Security and privacy model

The threat model is honest about what a local, single-user tool can and cannot promise, and the posture is layered.

- **Network exposure.** The server binds to `127.0.0.1` exclusively, and a

middleware rejects any request whose `Host` header is not a loopback name. Binding alone is not enough: without the Host check, a web page the user is merely visiting could POST to the local server via DNS rebinding and drive it from the outside. The two together close that vector. With no remote surface, there is deliberately no authentication layer to misconfigure.

- **Server-side request forgery.** The web-fetch path — used by both the agent

and deep research — refuses any URL that resolves to a private, loopback, or link-local address, and re-checks on every redirect hop, so a fetched page cannot bounce the agent onto localhost, the application's own API, or the local network.

- **Model-initiated actions.** The genuinely dangerous capabilities — arbitrary

shell, web fetch, and file writes outside the workspace — are each gated per call with no persistent allow. File access is deny-by-default outside the workspace plus explicit grants. Instructions embedded in fetched or retrieved text are framed as data, not commands.

- **Data at rest.** Conversations, settings, memories, indexed chunks,

documents, and provider API keys live in one local SQLite file. Keys are stored in plaintext — an OS-keychain integration is future work — but are masked (`•••1234`) in every API response, so the browser never re-receives a full secret. The honest consequence, stated plainly in the setup guide, is that anyone with read access to the data folder can read the keys; the folder should be treated as being as sensitive as the keys themselves.

- **Telemetry.** None. The application makes outbound connections only to the

model backends the user configured and to URLs the user, or the user's agent under the rules above, requests.

- **Residual risks.** A granted folder is fully readable and writable by any

model the user runs, including a poorly aligned local one; MCP servers run with the user's privileges and are trusted by the act of registration; and the unsigned executable requires a one-time SmartScreen click-through. These are documented rather than hidden, because a local tool's security story is only as good as its honesty about the edges.

16. Accessibility and theming

Appearance and colour vision are treated as two independent axes, so a colour-blind user is never forced to choose between an accessible palette and a preferred background. Appearance (system, light, or dark) and colour vision (default, protanopia, deuteranopia, tritanopia, achromatopsia) are applied as attributes on the document root and resolved entirely through CSS variable overrides, with a pre-render inline script setting them before first paint to avoid a flash of the wrong theme. Colour-vision modes remap only the four status hues and are tuned per background so status *text* clears WCAG AA contrast on both light and dark; achromatopsia drops hue entirely and relies on luminance. Underpinning all of it is a rule the UI follows regardless of palette: status is never conveyed by colour alone — every state also carries an icon and a text label — so meaning survives any colour transformation, and the palettes are a legibility improvement rather than a load-bearing signal. The application also honours the operating system's reduced-motion preference.

Accessibility extends past colour. Every interactive control carries an accessible name — icon-only buttons included, each labelled with the action and its target (for example, the specific conversation a delete button removes) — and the conversation and document lists, which are visually rows, are exposed as keyboard-focusable buttons with the active row marked. Actions that appear on hover — the per-message copy and edit controls, the per-code-block copy button — are faded rather than hidden outright, because a control hidden with `visibility` is removed from the tab order entirely and can never receive the focus that would reveal it; kept at zero opacity they remain focusable and appear on keyboard focus. The workspace is therefore navigable and operable with a screen reader and without a mouse, not only readable under an adapted palette.

17. Testing and assurance

A one-person codebase of this breadth stays trustworthy only with a testing discipline that matches. Every subsystem ships with an offline test suite that drives the *real* code paths against fakes rather than mocks of its own logic: a scripted fake provider exercises the agent and research loops through their actual event handling; `httpx.MockTransport` stands in for the Anthropic and Gemini wire protocols and for Ollama's native API; a deterministic fake embedder makes retrieval ranking meaningful without a model. Because the fakes sit only at the true boundaries — the network, the model — the suites verify the system's own behavior, and they need no network, no GPU, and no running model, so they run anywhere in seconds.

That property is what makes the suites enforceable rather than merely available. A single entry point runs every offline suite alongside the frontend's lint and typecheck, and continuous integration invokes that same entry point on every push and pull request, on Windows because that is the platform the application targets. The value is less in the automation than in the guarantee it removes from human memory: a regression in any subsystem fails the branch that introduced it, rather than waiting for whoever next thinks to run the suite by hand.

Beyond the suites, substantial features were put through an adversarial review before shipping: independent passes over the diff for correctness, security, frontend contract, and test coverage, with each finding challenged by skeptics before it was accepted, and every confirmed finding fixed and re-verified. Several of the correctness properties described earlier — the autosave flush-on-switch, the editor's offset lock, the research injection hardening, the cookbook's off-loop detection — are fixes that this process surfaced. The point is not that the code is flawless but that its most load-bearing behaviors have been

attacked on purpose.

18. Packaging and distribution

The release artifact is a portable zip: one PyInstaller onefile executable (windowed, over WebView2), the optional provider connectors, an end-user README, the setup guide, and the MIT license. Everything mutable lives beside the executable, so an installation is trivially copyable, movable, and deletable. On first run with an empty database, the server probes the well-known local backends — Ollama on `:11434`, LM Studio on `:1234` — and auto-configures any that respond, so a user who already runs a local model server reaches a working chat with zero configuration.

Version identity is stamped from a single source of truth, `APP_VERSION`, into the health endpoint, the UI footer, and the Windows file properties of the executable, so the three can never disagree. Two Windows-specific build lessons are preserved in the codebase for posterity: PyInstaller bundles are invisible to `pkgutil`, which is why builtin provider adapters are imported statically; and PowerShell 5.1 under `$ErrorActionPreference = "Stop"` turns a native command's harmless stderr into a terminating error, which is why the build scripts wrap native calls in `cmd /c "... 2>&1"`.

19. Limitations and roadmap

The honest limitations are the mirror image of the design choices. The application is single-user by intent and Windows-only in its packaging, though the server itself is portable Python. Multimodality is text-only. Key storage is plaintext on disk. And model *download* management is specific to Ollama.

Much of the original roadmap has since shipped and is described above: native Anthropic and Gemini connectors, deep research, the blind arena, retrieval over local files, and an AI writing editor, alongside agent memory, the hardware cookbook, and the accessibility work. The directions that remain, in rough order of value, are image (vision) input carried through the same normalized schema; an optional local token on the `/v1` hub for shared machines; OS-keychain key storage; and the largest gap relative to a full personal-assistant suite — productivity integrations such as email, calendar, and notes, each of which is effectively its own subsystem and would be evaluated on its own terms rather than folded in for completeness.

20. Conclusion

Syrudas AI is an argument, made in code, that a genuinely capable AI workspace — streaming chat, consent-gated agency, local retrieval, deep research, model comparison, a writing editor, protocol interoperability, editor integration, and one-click distribution — fits inside a small, single-maintainer codebase when one abstraction is chosen well and enforced everywhere. Each feature in this paper is less an invention than a consequence: route it through the normalized model and the provider contract, and history, export, streaming, and provider-independence come for free. Models will keep changing, faster than anything else in the stack. A workspace whose only firm opinion about models is a five-method contract is built to outlast every one of them.

Syrudas AI is open source under the MIT license. Architecture reference and setup: `README.md` *and* `docs/SETUP.md` *in the repository at github.com/LenSyrudas/syrudas.*